

# EE5203 L06 Lab Guide

*Button interrupts — replace polling with EXTI - Basri Erdogan*

**Mission:** Configure B1 (PC13) as a falling-edge interrupt so that one valid press toggles LD2 (PA5) — without reading the button anywhere in `while (1)`.

## Prepare before class

- ☐ Re-read the theory slides up to “The complete interrupt path.”
- ☐ Bring your working L05 HAL GPIO project.
- ☐ Know the answer to: why does pressing B1 produce a *falling* edge?

## Learning objectives

By checkoff time, you can:

1. configure an EXTI pin and enable its NVIC request in CubeMX;
2. prove with a breakpoint that the interrupt path reaches your callback;
3. pass an event from the callback to `main()` with a `volatile` flag;
4. debounce the button so one press produces exactly one event.

## Hardware and files

| Item           | Required value                                          |
|----------------|---------------------------------------------------------|
| Board          | Nucleo-F401RE (no wiring — B1 and LD2 are on the board) |
| Input          | B1 on PC13, external pull-up, <b>low while pressed</b>  |
| Output         | LD2 on PA5, high turns the LED on                       |
| Starting point | Copy of your L05 project, saved as L06_Button_Interrupt |

## Configure the interrupt in CubeMX

Open the `.ioc` file. Keep PA5 as GPIO\_Output (label LD2). Then:

1. Click PC13 and select GPIO\_EXTI13.
2. In **System Core** → **GPIO**: falling-edge interrupt mode, **no pull-up and no pull-down** (the board’s external resistor already holds PC13 high).
3. In **System Core** → **NVIC**: enable **EXTI line[15:10] interrupts** (PC13 is line 13; lines 10–15 share this entry). Leave the priority at its default.
4. Generate code, then verify the generated setup in `MX_GPIO_Init()`:

```
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

Write code only inside USER\_CODE blocks.

## Checkpoint A - The interrupt fires

Add below `main()` in the user-code section for functions:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
}
```

Build, flash, press B1 — the LED responds without any button code in the loop.

**Evidence for Checkpoint A:** a breakpoint on the `if` line is hit when you press B1, and `GPIO_Pin` equals `GPIO_PIN_13`. If execution never stops, recheck the PC13 mode and the NVIC checkbox.

## Checkpoint B - The callback reports, the loop acts

The LED action does not belong in the callback. In USER\_CODE BEGIN PV:

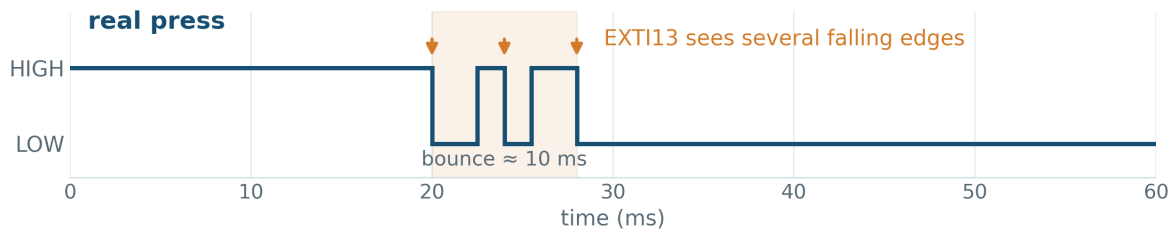
```
volatile uint8_t button_event = 0;
```

Change the callback body to only record the event (`button_event = 1;`), and let the main loop consume it:

```
if (button_event == 1) {
    button_event = 0;          /* consume before acting */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
}
```

## Checkpoint C - One press, one event

A real contact bounces — several falling edges per press:



Add `uint32_t last_press_ms = 0;` beside the flag and accept a press only when at least 80 ms has passed since the last accepted one:

```
uint32_t now = HAL_GetTick();
if ((now - last_press_ms) >= 80) {
    last_press_ms = now;
    button_event = 1;
}
```

The subtraction stays correct even when the millisecond counter wraps.

## Test behavior, not just compilation

| Test              | Expected observation  |
|-------------------|-----------------------|
| press once slowly | LD2 toggles once      |
| press ten times   | LD2 toggles ten times |
| hold B1 down      | no repeated toggling  |
| release B1        | no toggle             |

Record unexpected behavior *before* changing the code.

## Individual extension

Toggle LD2 only after **two** valid presses (a counter in the main loop); the interrupt and debounce code stay unchanged.

## Acceptance checklist

- ☐ .ioc shows PC13 = GPIO\_EXTI13, falling edge, no pull, NVIC enabled.
- ☐ Breakpoint proves HAL\_GPIO\_EXTI\_Callback runs on a press.
- ☐ `button_event` is declared `volatile`; the LED action lives in `main()`.
- ☐ Ten deliberate presses produce ten reliable state changes.
- ☐ Individual extension demonstrated.

## Individual oral check

Be ready for one question and one live change:

- Why does B1 use a falling edge and no internal pull?
- What runs first: `EXTI15_10_IRQHandler()` or your callback?
- Why must the callback stay short, and why is `volatile` required?
- What would happen without the 80 ms guard?

## Submit

Submit the project source and one debugger screenshot showing the breakpoint inside your callback.

## If you are stuck

Fix the **first** failed stage along the path B1/PC13 → EXTI13 → EXTI15\_10\_IRQn → handler → callback → `button_event` → main loop. **No callback**: PC13 mode / NVIC checkbox. **Wrong pin**: inspect `GPIO_Pin`. **Flag never seen**: missing `volatile`. **Several toggles**: 80 ms guard / `last_press_ms`. **LED dead**: PA5 output setup.